SDSU Machine Learning Course (EE510/EE600/CompE596)

Table of Contents

Programming Assignment: Neural Network 1
========== Part 1: Data Preprocessing ========== 1
========== Part 2: Train Neural Network ========== 3
========== Part 3: Evaluate performance ========== 6

Programming Assignment: Neural Network

```
Instructions
------------

Dataset comes from:
 http://networkrepository.com/pima-indians-diabetes.php

This file contains code that helps you get started on the
neural network assignment.

You will need to complete the following functions in this
assignment

   loadData.m
   featureNormalize.m
   trainNN.m
   evaluateAccuracy.m
   predict.m
   sigmoid.m

For this part of the assignment, you will need to change some
parts of the code below for various experiments (e.g., changing
learning rates).
```

```
% Initialization
clear ; close all; clc
```

========== Part 1: Data Preprocessing ==========

```
fprintf('===== Part 1: Data Preprocessing ===== \n');

% Instructions: The following code loads data into matlab, splits the
%               data into two sets, and performs feature normalization.
%               You will need to complete code in loadData.m, and
%               featureNormalize.m
```

```
% ================================================================



fprintf('Loading data ...\n');

% ====================== YOUR CODE HERE ======================
% Step 1: Load data
[X_train, y_train, X_test, y_test] = loadData();

% ================================================================
% Print out some data points
fprintf('First 10 examples from the training dataset: \n');
fprintf(' x = [%.0f %.0f %.0f %.0f %.0f %.0f %.0f %.0f], y = %.0f \n',...
    [X_train(1:10,:) y_train(1:10,:)]');
fprintf('\n');

[num_train, m] = size(X_train); % num_train is the number of total data
examples
% m is the number of features

% Scale features and set them to zero mean
fprintf('Normalizing Features ...\n');

% Step 2: Normalize the features.
% ====================== YOUR CODE HERE ======================
[Xn_train, mu, sigma] = featureNormalize(X_train);

% ================================================================
% Print out some normalized data points
fprintf('First 10 examples from the training dataset after normalization:
\n');
fprintf(' x = [%.0f %.0f %.0f %.0f %.0f %.0f %.0f %.0f], y = %.0f \n',...
    [Xn_train(1:10,:) y_train(1:10,:)]');
fprintf('\n');

fprintf('Program paused. Press enter to continue.\n');
pause;
```

```
===== Part 1: Data Preprocessing =====
Loading data ...
First 10 examples from the training dataset:
 x = [1 106 70 28 135 34 0 22], y = 0
 x = [2 155 52 27 540 39 0 25], y = 1
 x = [3 81 86 16 66 28 0 22], y = 0
 x = [4 197 70 39 744 37 2 31], y = 0
 x = [1 90 62 12 43 27 1 24], y = 0
 x = [1 93 56 11 0 22 0 22], y = 0
 x = [0 128 68 19 180 30 1 25], y = 1
 x = [0 105 84 0 0 28 1 62], y = 1
 x = [10 122 78 31 0 28 1 45], y = 0
 x = [0 137 68 14 148 25 0 21], y = 0

Normalizing Features ...
```

```
First 10 examples from the training dataset after normalization:
 x = [-1 -0 0 0 0 0 -1 -1], y = 0
 x = [-1 1 -1 0 4 1 -1 -1], y = 1
 x = [-0 -1 1 -0 -0 -1 -1 -1], y = 0
 x = [0 2 0 1 6 1 5 -0], y = 0
 x = [-1 -1 -0 -1 -0 -1 0 -1], y = 0
 x = [-1 -1 -1 -1 -1 -1 -0 -1], y = 0
 x = [-1 0 -0 -0 1 -0 3 -1], y = 1
 x = [-1 -0 1 -1 -1 -1 1 2], y = 1
 x = [2 0 0 1 -1 -1 0 1], y = 0
 x = [-1 1 -0 -0 1 -1 -1 -1], y = 0

Program paused. Press enter to continue.
```

========== Part 2: Train Neural Network ==========

```
fprintf('===== Part 2: Train Neural Network ===== \n');

% Instructions: The following code applies backpropagation algorithm to
%               estimate the parameters in a neural network model with
%               a single hidden layer.
%               You should complete code in trainNN.m, and sigmoid.m
%
%               Try running the backpropagation algorithm with
%               different values of alpha and num_hidden and see which
%               one gives you the best result.
% =============================================================


% ====================== YOUR CODE HERE ======================
% Step 1: Configure the hyper-parameters, including
%         1. alpha: learning rate
%         2. num_epochs: number of training epochs
%         3. num_hidden: number of hidden units (not including the bia unit)
%         4. num_output: number output unit

alpha = 0.1; % you should change this
num_epochs = 100; % you should change this
num_hidden = 5;   % you should change this
num_output = 1;   % you should change this

% initialize the weights: beta1 and beta2
beta1 = rand(num_hidden, m+1); % weights associated with links between input
and hidden layers
beta2 = rand(num_output, num_hidden+1); % weights associated with links
between hidden and output layers
% =============================================================

% ====================== YOUR CODE HERE ======================
% Step 2: Run backpropagation
[beta1, beta2, J_history] = trainNN(Xn_train, y_train, beta1, beta2, alpha,
```

```
num_epochs);

% =========================================================

% Plot the convergence graph
figure;
plot(1:numel(J_history), J_history, '-b', 'LineWidth', 2);
xlabel('Number of epochs');
ylabel('Cost J');
str = sprintf('num hidden = %d', num_hidden);
title(str);

% Display gradient descent's result
fprintf('beta1 computed from gradient descent: \n');
fprintf(' %f \n', beta1);
fprintf('\n');
fprintf('beta2 computed from gradient descent: \n');
fprintf(' %f \n', beta2);
fprintf('\n');

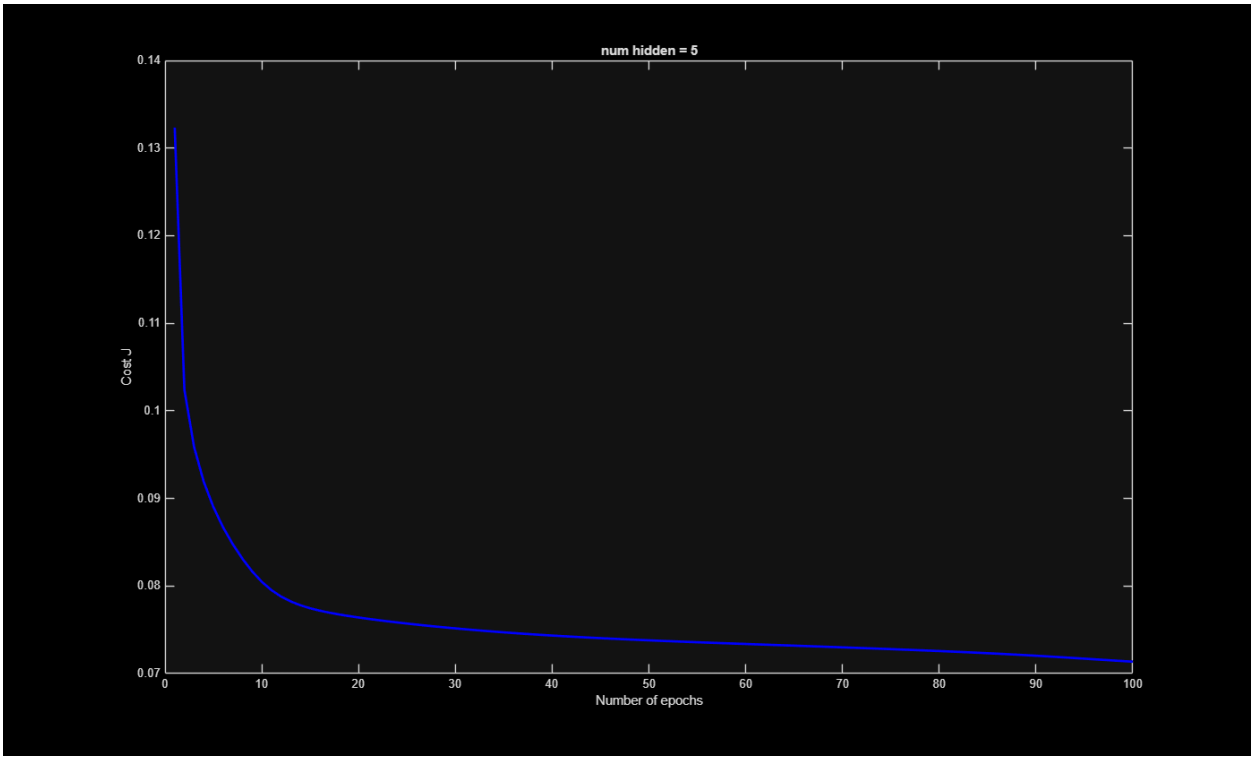
fprintf('Program paused. Press enter to continue.\n');
pause;
```

```
===== Part 2: Train Neural Network =====
beta1 computed from gradient descent:
 0.814572
 1.841100
 -1.801825
 0.420570
 2.001157
 -0.294471
 0.799960
 1.093857
 -0.154555
 0.476470
 1.686593
 1.216849
 1.115309
 0.582934
 -1.283858
 -0.116329
 -0.807200
 0.460837
 0.609440
 0.367972
 -1.299098
 1.500914
 0.526921
 0.318640
 0.378752
 -0.464864
 -0.553601
 0.819150
 0.321173
```

```
 0.363643
 0.098718
 0.668377
 1.249419
 0.103887
-0.540960
 0.698473
-0.777151
 2.438082
 0.062472
 0.708159
 2.642890
 1.855670
-0.985974
 0.696823
 0.617924

beta2 computed from gradient descent:
-2.083171
 1.834379
 2.015499
 2.388358
-0.516973
-2.458564

Program paused. Press enter to continue.
```



========== Part 3: Evaluate performance ==========

```
fprintf('===== Part 3: Evaluate Performance ===== \n');

% Instructions: The following code evaluates the performance of
%               the trained neural network model. You should
%               complete code in evaluateAccuracy.m, predict.m
% ============================================================


% ====================== YOUR CODE HERE ======================
% Hint: The testing set also needs to be normlized first
Xn_test= [(X_test - mu) ./ sigma]; % you should change this
    accuracy = evaluateAccuracy(beta1, beta2, Xn_test, y_test);

% ============================================================

% Display the prediction accuracy
fprintf('Accuracy:\n %f\n', accuracy);
fprintf('\n');

% ====================== YOUR CODE HERE ======================
% Step 2: Given a new input x = [3, 100, 79, 19, 100, 36, 0.8, 30], predict
the output
x_new = [3, 100, 79, 19, 100, 36, 0.8, 30];
x_new = [(x_new - mu) ./ sigma];

    y_new = predict(beta1,beta2,x_new); % You should change this

    % ============================================================
% display the predicted output
fprintf(['Predicted output:\n %d\n'], y_new);
```

```
===== Part 3: Evaluate Performance =====
Accuracy:
 81.304348

Predicted output:
 0
```

Published with MATLAB® R2026b